1. parallel Programming and Concurrency

Concurrency and parallelism both deal with handling multiple tasks, but they are different.

- Concurrency:

  Managing multiple tasks in overlapping time periods.
  Tasks run in an interleaved way but not necessarily at the same time.
  Improves system responsiveness.

- Parallel Programming:

  Tasks run simultaneously on multiple processors or CPU cores.
  Speeds up heavy computational work.

- Key Idea:

  Concurrency = dealing with many tasks.
  Parallelism = doing many tasks at the same time.

- Challenges:

  Task coordination, avoiding race conditions, and managing shared data safely.

2. Unit Testing and Test-Driven Development (TDD)

- Unit Testing:

  Tests a small unit of code (function or class) in isolation.
  Automated and repeatable.
  Helps detect bugs early and prevents regressions.
  Acts as documentation for expected behavior.

- Test-Driven Development (TDD):

  A development approach where tests are written before the code.

- TDD Cycle:

  1. Write a failing test.
  2. Write minimal code to pass the test.
  3. Refactor the code.
  4. Repeat.

- Benefits:

  Better understanding of requirements.
  Improved code quality and design.
  High test coverage.

- Limitations:

Writing tests takes extra time.
Poorly designed tests may reflect incorrect assumptions.

3. Asynchronous Programming (async / await)

Asynchronous programming allows applications to handle long-running operations without blocking the entire program.

- async and await:

  async defines a function that can pause and resume.
  await waits for an operation to complete while allowing other tasks to run.

- How It Works:

  Prevents freezing during slow operations.
  Uses futures/promises and internal state management.

- Why It Matters:

  Important for servers, web applications, and user interfaces.
  Improves performance and responsiveness when handling many requests.

12:51

Anyone

Post

والـ Lambda بقى لعبتها كبيرة

x => x * 2

الـ Lambda خلت الموضوع أسهل وأسرع جدًا.
وده اللي مخلي LINQ شغال بالشكل الجميل ده.

الخلاصة

لو فهمت Delegates صح...
إنت كده بدأت تدخل عالم الـ Functional Programming جوه #C.

وصدقني 👌
الموضوع مش مجرد ميزة في اللغة...
ده أسلوب تفكير بيخلي كودك أنضف وأذكى.

Rewrite with AI

12:50 22
Anyone
Post

```
        if (filter(n))
            Console.WriteLine(n);
    }
}
```

ونستخدمها كده:

```
ProcessNumbers(nums, x => x % 2 == 0);
ProcessNumbers(nums, x => x > 10);
```

شايف الفرق؟ 👀

الدالة بقت عامة... والسلوك هو اللي بنغيره.

إيه الفايدة من كل ده؟

✔ الكود بقى أنضف

✔ مفيش تكرار

✔ أسهل في التعديل

✔ مرن جدًا

بدل ما تعدل في الدالة كل شوية...

إنت بس بتغير الـ function اللى بتتبعت.

✦ Rewrite with AI

12:49

Anyone

Post

💡 مثال بسيط

❌ الطريقة التقليدية

```
public void PrintEvenNumbers(List<int> numbers)
{
    foreach (var n in numbers)
    {
        if (n % 2 == 0)
            Console.WriteLine(n);
    }
}
```

المشكلة هنا إن الدالة مربوطة بالأرقام الزوجية بس.

✅ باستخدام Delegate

```
public void ProcessNumbers(List<int> numbers, Func<int, bool> filter)
{
    foreach (var n in numbers)
```

✦ Rewrite with AI

12:49

Anyone

Post

تبعت دالة كـ parameter

تخزن دالة في متغير

ترجع دالة من دالة

وده بالظبط مفهوم إن الـ Functions تبقى
First-Class Citizens 👏

🧩 طيب ده فين بقى الـ Functional
Programming؟

بتقولك functional programming:

> متربطش السلوك جوه الكود... خلي السلوك بتبعت
كقيمة.

يعني بدل ما تكتب دالة مخصوص لكل حالة،
تكتب دالة عامة... وتبعتلها المنطق اللي هي تشتغل
بيه.

✦ Rewrite with AI

12:49

Anyone

Post

يعني إيه Delegates ومالهم بالـ Functional Programming في C#؟

وأنت شغال بـ C# أكيد عدي عليك مصطلح Delegate...
بس هل فكرت قبل كده إن الـ Delegate هو اللي بيخليك تطبق مفهوم الـ Functional Programming جوه لغة أصلاً معمولة OOP؟

تعالى نبسطها 👇

🔍 الأول... يعني إيه Delegate؟

ببساطة كده 👇
الـ Delegate هو متغير بيشيل دالة جواه.

آه زي ما قرأت كده 👌
بدل ما المتغير يشيل رقم أو نص، يقدر يشيل Function.

وده معناه إنك تقدر:

✦ Rewrite with AI